Deployment of an SDN-based UAV Network: Controller Placement and Tradeoff between Control Overhead and Delay

Shams ur Rahman
School of Electronics Engineering, Kyungpook National University, Daegu, Korea
shamsuet@gmail.com

Geon-Hwan Kim
School of Electronics Engineering, Kyungpook National University, Daegu, Korea
kgh76@ee.knu.ac.kr

You-Ze Cho
School of Electronics Engineering, Kyungpook National University, Daegu, Korea
yzcho@ee.knu.ac.kr

Ajmal Khan
COMSATS Institute of Technology Attock, Pakistan
engajmal@yahoo.com

***Abstract*— This paper investigates the deployment of an SDN-based UAV network providing communication service. Such a network is extremely useful in emergency scenarios. In such a network, the mobility of UAVs is minimal. We consider the placement of the SDN controller and its implications in terms of communication overhead and end-to-end delay of the control packets exchanged between the UAVs and the controller. We also propose a scheme for reducing the overhead caused by the control packets, sent by each forwarding element (UAV) to the controller. We evaluate the performance of the proposed scheme using ns-2. Simulation results show that the controller should be placed in a central location to minimize the number of hops and hence reduce the delay experienced by control packets. Moreover, simulation results show that a tradeoff exists between control packet overhead and their end-to-end delay. Attempting to reduce the overhead causes the end-to-end delay to increase.**

***Keywords*— SDN, UAV Network, physical deploment, SDN controller placement.**

I. Introduction

Recently Unmanned Aerial Vehicle (UAV) based communications systems have attracted increasing research and commercial interest [1-3]. In such networks, UAVs carry the equipment, such as WiFi access point, to provide communication service. UAV-based communication networks are extremely useful in emergency scenarios, such as post-disaster scenario. Unlike many other UAV applications, such as search and rescue, the mobility of a UAV network providing communication service is minimal, resulting in a relatively stable network topology. On the network infrastructure side, Software Defined Networking (SDN) has emerged as a promising alternative to the traditional IP networks [4]. SDN separates the network logic (control plane) from the data plane. SDN consists of a central controller and forwarding elements (switches and routers). The centralized control makes SDN ideal for a UAV network because UAV networks are usually of small size. Furthermore, all nodes in a UAV network cooperate towards a common goal; therefore, every node (UAV) is willing to share control information with the SDN controller.

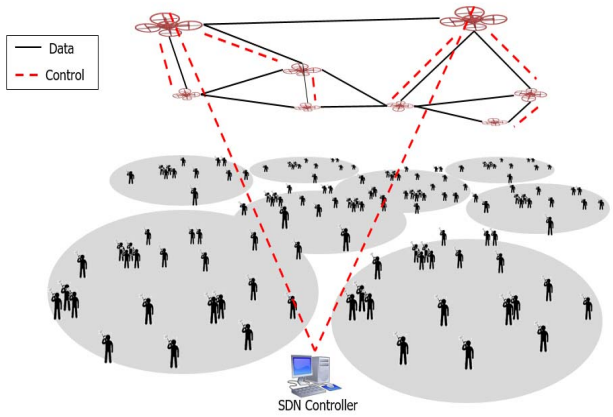

Figure 1. A Typical SDN-Based UAV Network

In SDNs, the controller gathers network statistics from switches, installs flows on switches and sends out administrative policies to the switches [5]. The controller also needs to know the latest network topology. Therefore, connectivity between the SDN controller and the UAVs is extremely important. A key aspect of a deployment method is the placement of the SDN controller [6].

Fig. 1 shows a typical scenario of an SDN-based UAV network. The UAVs are acting as forwarding elements and the SDN-controller is managing the network. Each UAV is providing communication to users within its transmission range.

In this paper, we explore the deployment of an SDN-based UAV network. We also discuss the issue of overhead caused by the UAV-SDN controller communications and propose solution for reducing it. We evaluate the performance of our proposed scheme using ns-2.

II. Control and Data Packet (User Traffic) Communication in an SDN-based UAV Network

In SDN-based networks, the SDN controller is the brain of the network. It is the strategic control point, managing flow control to forwarding elements and performing a variety of other control functions. Therefore, all control information flows from the forwarding elements (UAVs) to the controller or from the controller to the UAVs. For example, when a new data session starts (or in SDN terms, a new flow arrives), the SDN controller needs to be informed about the source and destination of the session. The SDN controller computes the routing path based on some criteria and installs forwarding rules on the forwarding elements (UAVs). The SDN controller can only

Authorized licensed use limited to: University of Zululand. Downloaded on March 25,2026 at 01:09:51 UTC from IEEE Xplore. Restrictions apply.

determine a path for a flow if it knows the network topology. Therefore, in an SDN-based UAV network, each UAV has to communicate topology related information (such as a list of its neighbors) to the controller on regular basis. Moreover, if a change in the network topology occurs (a UAV notices changes in its neighborhood), it has to be communicated to controller immediately. Moreover, the SDN-controller might query the UAVs for various statistical information such as link capacity etc. and the UAVs have to send the queried statistical information to the controller. Therefore, exchange of control information between the controller and the UAVs is critical for the operation of an SDN-based UAV network. Data packet are, however, routed by the UAVs according the forwarding rules installed by the controller.

III. MULTIHOP COMMUNICATION WITH CONTROLLER

As is obvious from the discussion in Section II, that exchange of control messages between the SDN controller and the UAVs is extremely important. Direct communication between the controller and the UAVs is not practical for a number of reasons such as the requirement to transmit at high power. Therefore, multi-hop communication has to be used.

The controller can be positioned anywhere near the UAV network. Ideally it should be placed at some central location to minimize the number of hops to the farthest UAVs. As shown by the dashed lines in Fig. 1, UAVs close to the controller can send their control packets directly to the controller; whereas, UAVs far away from the controller can get their packet relayed by other UAVs closer to the controller than themselves. From the UAVs perspective, this method is energy efficient, because, unlike direct communication with controller, they are not required to transmit at high power. This advantage makes it highly attractive for practical scenarios as UAVs are usually powered by batteries of limited capacity.

However, this method has its complexities. Packets exchanged between UAVs and the controller will experience delay. Due to multi-hop communication, each control packet will be transmitted multiple times (once by each hop) resulting in high communication overhead as compared to direct communication between UAVs and the controller. Therefore, we investigate these problems and propose some solutions.

Communication overhead and delay

As discussed in previous sections that the SDN controller maintains information about the network topology, the traffic loads on each link, the traffic demands of each node and various other statistics about the network. Therefore, each switch (UAV) has to send updates about the network status and events to the controller. In a UAV network, certain types of information, such as topology, has to be regularly sent to the controller so that the controller has the up-to date view of the network. Sending update information over multiple hops can cause significant overhead if not properly dealt with.

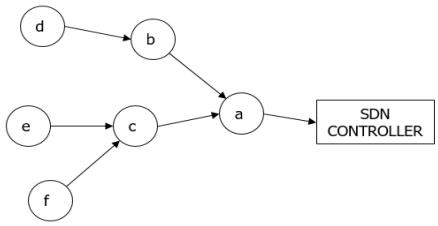

Figure 2. Flow of control (update) packets from UAVs to SDN controller

Consider Fig. 2, showing the flow of control packets from UAVs to SDN controller. Assuming each UAV has to send an update (control) packet to the controller after a certain period of time; we will call this an *update round*. If the update packet from each UAV is forwarded separately (no merging with the update packets from other UAVs), then there will be a total of 14 transmissions for each update round. For example, each update packet from *d* will be transmitted three times: first by *d*, then by *b* and then by *a*. However, the number of transmission per update round can be reduced from 14 to 6 if the update packets are merged along the way. For example, when *b* receives update packet from *d*, it merges its own packets with it and then transmits to *a*. Similarly, *c* will merge the update packets form *e* and *f* with its own and then send to *a*. Lastly, *a* will merge its own update information with the packets received from *b* and *c* and then transmit it to the controller.

The merging strategy can reduce the overhead significantly, but it also has a downside: packets will experience more delay now. This is because each UAV has to buffer the update packets it receives and when update packets from all its upstream neighbors are received, they are merged with its own update packet and sent to the next hop.

One might argue that merging control packets results in larger packets and hence the reduction in overhead is cancelled. This, however, is not completely true. First, only the payload of control packet needs to be merged and hence the header overhead is reduced. Second, transmission of each packet has its associated overhead such as the exchange of RTS/CTS and ACK frames in IEEE802.11. With merging, this overhead is reduced.

A tradeoff between the overhead and delay can be achieved by imposing a maximum buffering time limit, BuffIntrvl, on the update packets. No UAV will buffer an update packet from

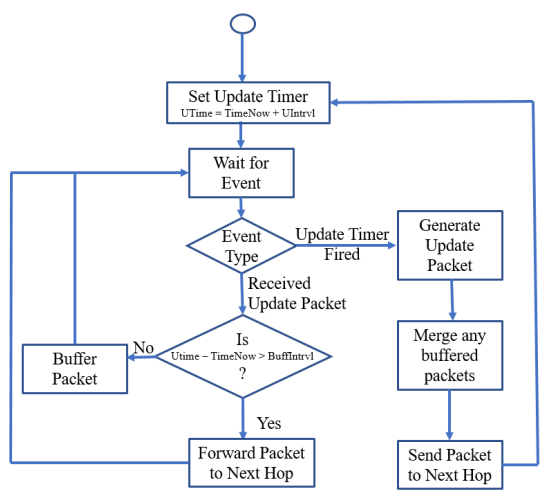

Figure 3. Flowchart for handling update packets by each UAV to reduce overhead

Authorized licensed use limited to: University of Zululand. Downloaded on March 25,2026 at 01:09:51 UTC from IEEE Xplore. Restrictions apply.

upstream nodes for a time longer than BuffIntrvl. Fig. 3 shows a flowchart showing the procedure for handling update packets by each UAV. Each UAV sets a timer for sending a periodic update packet to the controller. If a UAV receives an update packet from its upstream nodes, the UAV buffers the received packet if there is less than BuffIntrvl time remaining in generating its own update packet. Otherwise the UAV forwards the received packet to the next hop towards the controller. When update timer of the UAV fires, an update packet is generated, then any buffered packets from other UAV are merged and then sent to the next hop towards the controller.

IV. Simulation Results

The procedure shown in Fig. 3 was implemented using ns-2 and experiments for various values of BuffIntrvl were carried out. A total of 36 nodes (UAVs) were deployed on equally spaced grid points in an area of 1000×1000 m^2. The controller was positioned at a corner for of the deployment area. The transmission range was maintained at 200 m. The periodic interval was set to 1 sec for each UAV. Update packets to the controller were routed using the shortest path. Fig. 4 sows the delay experienced by update packets for different number of hops. The delay increases with the number of hops. Equation (1), obtained by curve fitting the data in MATLAB, nicely fits the data points in Fig. 5, where d represents the delay and h represents the number of hops. This indicates that the controller should be placed in some central location so that the maximum number of hops is minimized.

$$d = 0.001h^4 + 0.011h^3 - 0.0556h^2 + 0.1181h - 0.0775 \quad (1)$$

Fig. 5 shows the effect of buffering time (that is BuffIntrvl) on update packet overhead and the delay experienced update packets before reaching the controller. As the buffering time increases, the overhead drops but the delay increases. For larger buffering times, the probability of merging update packets intermediate UAV is high therefore, the number of transmissions are reduced. The relationship between buffering time and delay is straightforward: the longer a packet waits in buffers, the longer it will take to reach the controller. It is worth mentioning here that there are a variety of control packets exchanged between UAVs and the controller. Some types of control packets, such flow setup request, are more sensitive delay than others. Therefore, different buffering times can be specified for different types of control packets based on their sensitive to delay.

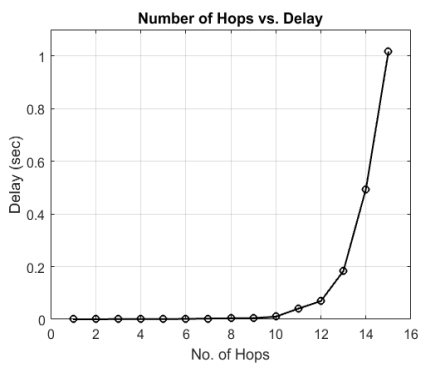

Figure 4. Number of Hops vs. Delay

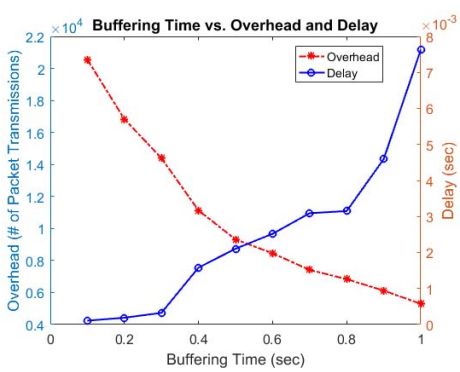

Figure 5. Buffering Time vs. Overhead and Delay

V. Conclusions

Deployment an SDN-based UAV network was discussed. In particular, the placement of SDN controller was investigated. Results indicate that the SDN controller should be placed in a central location to minimize the number of hops from the UAVs. Furthermore, control packet overhead and end-to-end delay issues were discussed. A scheme was proposed for achieving a tradeoff between overhead and delay. The scheme involved buffering of update packets at intermediate nodes and merging of control packets. It was found that as the buffering time increases, the overhead decreases but the delay increases.

VI. Acknowledgement

This research was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIP) [NRF-2015R1D1A1A01059623].

References

[1] S. Hayat, E. Yanmaz, and R. Muzaffar, "Survey on Unmanned Aerial Vehicle Networks for Civil Applications: A Communications Viewpoint," *IEEE Communications Surveys & Tutorials,* vol. 18, pp. 2624-2661, 2016.

[2] *Loon Project - Baloon-Powered Internet for Everyone*. [Online]. Available: https://x.company/loon/. [Accessed: Apr. 24, 2017].

[3] *Flying Aquila: Early lessons from the first full-scale test flight and the path ahead*. [Online]. Available: https://code.facebook.com/posts/268598690180189/flying-aquila-early-lessons-from-the-first-full-scale-test-flight-and-the-path-ahead/. [Accessed: Apr. 24, 2017].

[4] D. Kreutz, F. M. V. Ramos, P. E. Veríssimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-Defined Networking: A Comprehensive Survey," *Proceedings of the IEEE,* vol. 103, pp. 14-76, 2015.

[5] O. Awobuluyi, "Periodic Control Update Overheads in OpenFlow-Based Enterprise Networks," in *2014 IEEE 28th International Conference on Advanced Information Networking and Applications*, 2014, pp. 390-396.

[6] B. Heller, R. Sherwood, and N. McKeown, "The controller placement problem," presented at the Proceedings of the first workshop on Hot topics in software defined networks, Helsinki, Finland, 2012.

Authorized licensed use limited to: University of Zululand. Downloaded on March 25,2026 at 01:09:51 UTC from IEEE Xplore. Restrictions apply.